FINAL PROJECT

Session 2025 – 2029

Section D

Submitted by:

Moghees Mohiuddin 2025-CS-210

Ahsan Mustafa 2025-CS-197

Submitted to:

Sir Hamza Farooq

Course:

Programming Fundamentals Lab

University of Engineering & Technology, Lahore

Introduction

This project is a C++ console-based **Cargo Management System** developed using a menu-driven approach to provide an interactive and user-friendly experience. The system manages basic cargo operations through a text-based interface and is divided into two main interfaces: **Consigner and Company**.

The **Consigner interface** allows customers to enter cargo details, book shipments, and view shipment information, while the **Company interface** is used to manage cargo records, track consignments, and oversee overall operations. The project demonstrates how real-world logistics processes can be modeled using fundamental C++ programming concepts.

Objective

The main objective of this project was to practically implement all the concepts learned in the C++ programming lab. These include **menu-driven programming, functions, conditional statements, loops, arrays/structures, pointers, and exception handling**. The project aims to strengthen logical thinking, improve problem-solving skills, and show how theoretical knowledge can be applied to build a complete working management system.

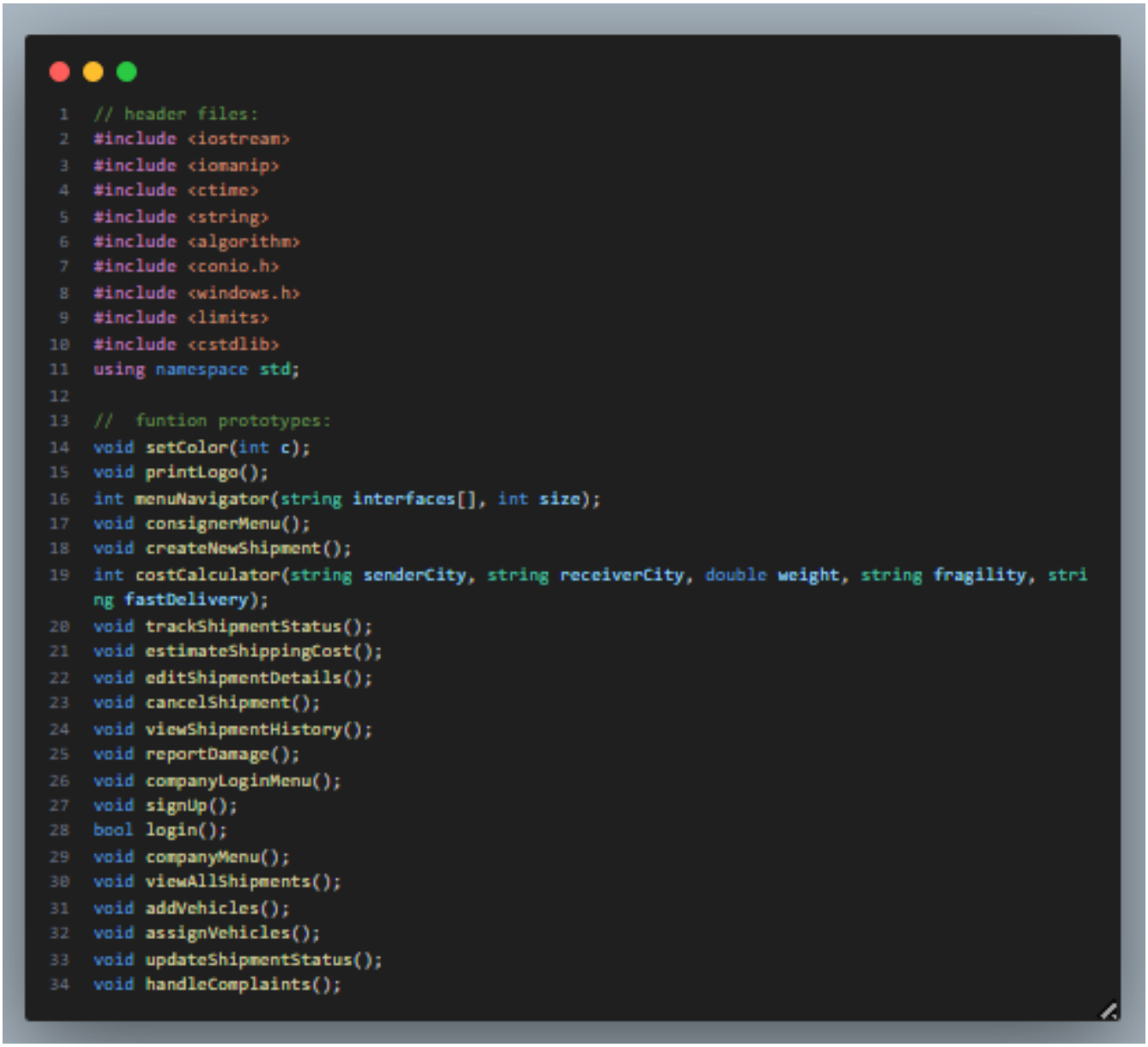
System Features

- **Beautiful UI**
- **User-Friendly navigation**
- **Proper Tracking**
- **Easy to use**
- **Console-Based**

CODE EXPLANATION

1. Header Files Inclusion

This screenshot shows all the header files used in the project to support input/output handling, string processing, formatting, validation, and Windows-based console operations.



```
// header files:
#include <iostream>
#include <iomanip>
#include <ctime>
#include <string>
#include <algorithm>
#include <conio.h>
#include <windows.h>
#include <limits>
#include <cstdlib>
using namespace std;

//  funtion prototypes:
void setColor(int c);
void printLogo();
int menuNavigator(string interfaces[], int size);
void consignerMenu();
void createNewShipment();
int costCalculator(string senderCity, string receiverCity, double weight, string fragility, stri
ng fastDelivery);
void trackShipmentStatus();
void estimateShippingCost();
void editShipmentDetails();
void cancelShipment();
void viewShipmentHistory();
void reportDamage();
void companyLoginMenu();
void signUp();
bool login();
void companyMenu();
void viewAllShipments();
void addVehicles();
void assignVehicles();
void updateShipmentStatus();
void handleComplaints();
```

2. Main function & Project Logo Screen

The main function calls 4 built-in functions.

The printLogo function displays the animated ASCII logo and tagline of the Cargo Management System.

```
// main function:
int main() {
    printLogo();
    string interfaces[3] = {"Coonsigner", "Company", "Exit"};


    while (true) {

        int choice = menuNavigator(interfaces, 3);

        if (choice == 0) {
            consignerMenu();
        }
        else if (choice == 1) {
            companyLoginMenu();
        }
        else if (choice == 2) {
            exit(0);
        }
    }


    return 0;
}


// function to print the static logo:
void printLogo() {
    system("cls");
    setColor(5);  // purple color
    cout<<"::::           ::::        ::::           ::::::::::::    \n"; Sleep(100);
    cout<<":::::::::    :::::::::      :::::::::      :::::::::    ::::::::: \n"; Sleep(100);
    cout<<"::::::::::::::::::::::    ::::::::::::    ::::               \n"; Sleep(100);
    cout<<"::::    ::::    ::::   ::::       ::::   ::::               \n"; Sleep(100);
    cout<<"::::            :::: ::::::::::::::::::: ::::               \n"; Sleep(100);
    cout<<"::::            :::: ::::           :::: ::::               \n"; Sleep(100);
    cout<<"::::            :::: ::::           :::: ::::               \n"; Sleep(100);
    cout<<"::::            :::: ::::           :::: ::::               \n"; Sleep(100);
    cout<<"::::            :::: ::::           :::: ::::               \n"; Sleep(100);
    cout<<"::::            :::: ::::           ::::  ::::::::    ::::::::\n"; Sleep(100);
    cout<<"::::            :::: ::::           ::::      ::::::::::::     \n";Sleep(100);
    cout << endl;
    cout<<"                YOUR CARGO, OUR RESPONSIBILTY                \n";Sleep(300);
    setColor(7);
    cout << endl;

    Sleep(1300);
    system("cls");
}
```

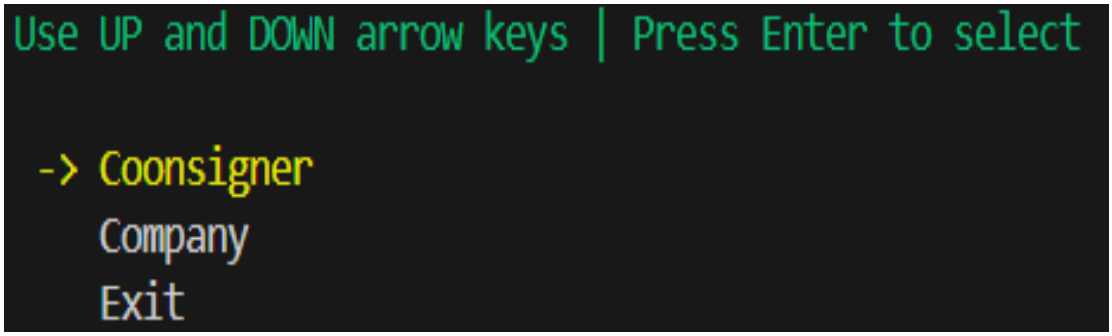
Output

3. Main Menu (Consigner / Company / Exit)

This menu allows users to navigate between consigner services, company login, or exit the program using arrow-key navigation and select by pressing enter.

```cpp
// UI
void setColor(int c) {
    SetConsoleTextAttribute(GetStdHandle(STD_OUTPUT_HANDLE), c);
}


// function to display menus with arrow nav bar:
int menuNavigator(string interfaces[], int size){
    int choice = 0;
    char key;

    system("cls");
    while (true) {

        setColor(2);
        cout << "Use UP and DOWN arrow keys | Press Enter to select\n" << endl;
        setColor(7);

        for (int i = 0; i < size; i++) {
            if (i == choice) {
            setColor(6);
            cout << " -> " + interfaces[i] << endl;
            setColor(7);
            }
            else
                cout << "    " + interfaces[i] << endl;
        }

        key = getch();

        if (key == 72) {
            choice = (choice - 1 + size) % size;
        }
        else if (key == 80) {
            choice = (choice + 1) % size;
        }
        else if (key == 13) {
            return choice;
        }

        system("cls");
    }
}
```

Output



4. Consigner Menu Interface

This screen displays all consigner functionalities including shipment creation, tracking, cost estimation, editing, cancellation, history viewing, and damage reporting.

```
//  function to display consigner menu:
void consignerMenu() {
    string consignerMenu[8] = {"Create new shipment", "Track Shipment status", "Estimate Shipping Cost",
                               "Edit Shipment Details", "Cancel Shipment", "View Shipment History", "Report a damage", "Back"};



    // displaying consigner menu:
    while (true) {

        int choice = menuNavigator(consignerMenu, 8);
        system("cls");
        setColor(1);
        setColor(7);

        if (choice == 0) {
            createNewShipment();
        }
        else if (choice == 1) {
            trackShipmentStatus();
        }
        else if (choice == 2) {
            estimateShippingCost();
        }
        else if (choice == 3) {
            editShipmentDetails();
        }
        else if (choice == 4) {
            cancelShipment();
        }
        else if (choice == 5) {
            viewShipmentHistory();
        }
        else if (choice == 6) {
            reportDamage();
        }
        else if (choice == 7) {
            break;
        }
    }
}

// arrays to store shipment details:
string cargoTypes[50], senderNames[50], senderContacts[50],
       receiverNames[50], receiverContacts[50], receiverAddresses[50],
       senderCities[50], receiverCities[50], fragilities[50],
       fastDeliveries[50], shipmentStatus[50], trackingIDs[50], bookingDatesTimes[50];

double weights[50], shipmentCosts[50];
int shipmentCount = 0;
```

Output

```
Use UP and DOWN arrow keys | Press Enter to select

 -> Create new shipment
    Track Shipment status
    Estimate Shipping Cost
    Edit Shipment Details
    Cancel Shipment
    View Shipment History
    Report a damage
    Back
```

5. Create New Shipment – Initial Input Screen

This interface collects cargo type and sender/receiver basic information required to book a new shipment, define arrays globally and after taking input stores them in their array and then finds the estimate cost using the built-in function.

```cpp
// function to create a new shipment:
void createNewShipment() {
    string cargoType, senderName, senderContact,
           receiverName, receiverContact, receiverAddress,
           senderCity, receiverCity, fragility, fastDelivery;
    double weight;

    cout<< "\n\033[34m---------------  BOOK YOUR SHIPMENT  ----------------\033[0m\n";

    cin.ignore(numeric_limits<streamsize>::max(), '\n'); // flush buffer

    cout << "Enter the Cargo Type: ";
    getline(cin, cargoType);

    cout << "Enter the Sender's Name: ";
    getline(cin, senderName);

    cout << "Enter the Sender's Contact: ";
    getline(cin, senderContact);

    cout << "Enter the Receiver's Name: ";
    getline(cin, receiverName);

    cout << "Enter the Receiver's Contact: ";
    getline(cin, receiverContact);

    cout << "Enter the Receiver's Address: ";
    getline(cin, receiverAddress);

    cout << "Enter the Sender's City: ";
    getline(cin, senderCity);

    cout << "Enter the Receiver's City: ";
    getline(cin, receiverCity);

    // handling exceptions for weight input
    do {
        cout << "Enter the weight (in kilograms): ";
        cin >> weight;

        if (weight <= 0 || cin.fail() ) {
            cout << "\033[31mWeight must be a positive number. Please try again.\033[0m" << endl;

            cin.clear(); // clear error flag
            cin.ignore(numeric_limits<streamsize>::max(), '\n'); // discard invalid input
            weight = -1; // reset weight to stay in loop

        }
    } while (weight <= 0);

    cout << "Is the shipment Fragile? (Yes/No): ";
    cin >> fragility;

    cout << "Do you want Fast Delivery? (Yes/No): ";
    cin >> fastDelivery;

    // shipment status
    shipmentStatus[shipmentCount] = "Pending";
```

```cpp

    // storing shipment details in arrays
    cargoTypes[shipmentCount] = cargoType;
    senderNames[shipmentCount] = senderName;
    senderContacts[shipmentCount] = senderContact;
    receiverNames[shipmentCount] = receiverName;
    receiverContacts[shipmentCount] = receiverContact;
    receiverAddresses[shipmentCount] = receiverAddress;
    senderCities[shipmentCount] = senderCity;
    receiverCities[shipmentCount] = receiverCity;
    weights[shipmentCount] = weight;
    fragilities[shipmentCount] = fragility;
    fastDeliveries[shipmentCount] = fastDelivery;

    cout << "\nShipment has been created successfully! Press any key to proceed." << endl;
    getch();

    // generating Tracking ID
    trackingIDs[shipmentCount] = "MAC" + to_string(shipmentCount);
    cout << "Your tracking number is: " << trackingIDs[shipmentCount] << endl;

    // saving estimated cost in array
    shipmentCosts[shipmentCount] = costCalculator(senderCity, receiverCity, weight, fragility, fastDelivery);
    cout << "Your Estimated Cost is: " << shipmentCosts[shipmentCount] << " PKR" << endl;
    getch();

    shipmentCount++;
}
```

Output

```
===============  BOOK YOUR SHIPMENT  =================

Enter the Cargo Type: Mangoes
Enter the Sender's Name: Ahsan Mustafa
Enter the Sender's Contact: 0317-7050444
Enter the Receiver's Name: Moghees Mohiuddin
Enter the Receiver's Contact: 0342-2444427
Enter the Receiver's Address: 18, Ghaziabad, Islampura
Enter the Sender's City: Fort Abbas
Enter the Receiver's City: Lahore
Enter the weight (in kilograms): 10
Is the shipment Fragile? (Yes/No): No
Do you want Fast Delivery? (Yes/No): Yes

Shipment has been created successfully! Press any key to proceed.
Your tracking number is: MAC0
Your Estimated Cost is: 1700 PKR
```

6. Cost calculator

This function calculates the cost by taking parameters, it also uses pointer to store the cost and then it returns back.

```
// function for cost calculation:
int costCalculator(string senderCity, string receiverCity, double weight, string fragility, string fastDelivery) {
    double * estimatedCost = new double;  // using pointer for estimated cost
    *estimatedCost = 0;
    // base cost calculation
    if (senderCity == receiverCity) {
        *estimatedCost = weight * 120;
    }
    else if (senderCity != receiverCity) {
        *estimatedCost = weight * 150;
    }

    // additional charges if fragile or fast delivery:
    if (fragility == "Yes")
        *estimatedCost += 70;
    if (fastDelivery == "Yes")
        *estimatedCost += 200;

    double cost = *estimatedCost;
    delete estimatedCost;   // to avoid memory leak.
    return cost;
}
```

7. Tracking shipment

This function checks the tracking number and prints the current status.

```
// function to track shipment:
void trackShipmentStatus() {

    cout<< "\n\033[34m===============  TRACK YOUR SHIPMENT  ================\033[0m\n";

    string trackingNumber;
    cout << "Enter your tracking number: ";
    cin >> trackingNumber;

    bool found = false;
    for (int i = 0; i < shipmentCount; i++) {
        if (trackingIDs[i] == trackingNumber) {
            // printing shipment status
            cout << "\nStatus:\033[1m" << shipmentStatus[i] <<"\033[0m" << endl;
            found = true;
            break;
        }
    }
    if (!found) {
        cout << "\033[31mNo shipment found with the given tracking number.\033[0m" << endl;
    }
    getch();
}
```

Output

```
===============  TRACK YOUR SHIPMENT  =================
Enter your tracking number: MAC1

Status:Pending
```

8. Estimate Shipping Cost

This function calculates the estimated shipping cost using the built-in function costCalculator.

```
// function to estimate shipment cost:
void estimateShippingCost() {

    string senderCity, receiverCity, fragility, fastDelivery;
    double weight;

    cout<< "\n\033[34m===============  COST CALCULATOR  ================\033[0m\n";

    cout << "Enter the city of sender: ";
    cin >> senderCity;

    cout << "Enter the city of receiver: ";
    cin >> receiverCity;

    cout << "Is the shipment Fragile? (Yes/No): ";
    cin >> fragility;

    cout << "Do you want Fast Delivery? (Yes/No): ";
    cin >> fastDelivery;

    do
    {
        cout << "Enter the weight (in kg): ";
        cin >> weight;
        if (weight <= 0 || cin.fail()) {
            cout << "\033[31mWeight must be a positive number. Please try again.\033[0m" << endl;

            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            weight = -1;; // reset weight to stay in loop
        }
    } while (weight <= 0);

    // printing estimated cost
    cout << "\nEstimated Cost: " << costCalculator(senderCity, receiverCity, weight, fragility, fastDelivery) << " PKR" << endl;

    getch();
}
```

Output

```
===============  COST CALCULATOR  =================
Enter the city of sender: Multan
Enter the city of receiver: Karachi
Is the shipment Fragile? (Yes/No): Yes
Do you want Fast Delivery? (Yes/No): Yes
Enter the weight (in kg): 12

Estimated Cost: 2070 PKR
```

9. View Shipment History

This function prints all the shipments.

```
// function to fetch shipment history:
void viewShipmentHistory() {

    // printing shipment history
    cout << "\n\033[34m==============  Shipment History  ================\033[0m\n" << endl;
    if (shipmentCount == 0) {
            cout << "\033[31mNo shipments available.\033[0m" << endl;
            getch();
            return;
    }

    for (int i = 0; i < shipmentCount; i++) {
        cout << "Consigner Shipment " << i + 1 << ":" << endl;
        cout << "--------------------------------------------------" << endl;

        cout << left; // align text to the left

        cout << "  " << setw(20) << "Tracking Number:" << trackingIDs[i] << endl;
        cout << "  " << setw(20) << "Sender Name:" << senderNames[i] << endl;
        cout << "  " << setw(20) << "Receiver Name:" << receiverNames[i] << endl;
        cout << "  " << setw(20) << "Sender's City:" << senderCities[i] << endl;
        cout << "  " << setw(20) << "Receiver's City:" << receiverCities[i] << endl;
        cout << "  " << setw(20) << "Receiver's Address: " << receiverAddresses[i] << endl;
        cout << "  " << setw(20) << "Weight:" << weights[i] << " kg" << endl;
        cout << "  " << setw(20) << "Fragile:" << fragilities[i] << endl;
        cout << "  " << setw(20) << "Fast Delivery:" << fastDeliveries[i] << endl;
        cout << "  " << setw(20) << "Estimated Cost:" << shipmentCosts[i] << " PKR" << endl;
        cout << "  " << setw(20) << "Shipment Status:" << "\033[1m" << shipmentStatus[i] << "\033[0m" << endl;

        cout << "--------------------------------------------------\n" << endl;
    }
        getch();
    }
```

Output

```
==============  Shipment History  ================

Consigner Shipment 1:
--------------------------------------------------
  Tracking Number:    MAC0
  Sender Name:        Ahsan Mustafa
  Receiver Name:      Moghees Mohiuddin
  Sender's City:      Fort Abbas
  Receiver's City:    Lahore
  Receiver's Address: 18, Ghaziabad, Islampura
  Weight:             10 kg
  Fragile:            No
  Fast Delivery:      Yes
  Estimated Cost:     1700 PKR
  Shipment Status:    Pending
--------------------------------------------------

Consigner Shipment 2:
--------------------------------------------------
  Tracking Number:    MAC1
  Sender Name:        Kamran Siddique
  Receiver Name:      Zain Abbas
  Sender's City:      Sheikhupura
  Receiver's City:    Lahore
  Receiver's Address: 88, Shahdara Town
  Weight:             25 kg
  Fragile:            Yes
  Fast Delivery:      No
  Estimated Cost:     3820 PKR
  Shipment Status:    Pending
--------------------------------------------------

Consigner Shipment 3:
--------------------------------------------------
  Tracking Number:    MAC2
  Sender Name:        Noman Iqbal
  Receiver Name:      Ayesha Noor
  Sender's City:      Rawalpindi
  Receiver's City:    Rawalpindi
  Receiver's Address: Satellite Town
  Weight:             5 kg
  Fragile:            No
  Fast Delivery:      No
  Estimated Cost:     600 PKR
  Shipment Status:    Pending
--------------------------------------------------
```

10. Edit Shipment Details

This function checks the tracking number and then retakes the inputs and replace the old ones with new one.

```
// function to edit shipment details:
void editShipmentDetails() {
    cout<< "\n\033[34m==============  EDIT SHIPMENT DETAILS  ================\033[0m\n";
    if (shipmentCount == 0) {
        cout << "\033[31mNo shipments available.\033[0m" << endl;
        getch();
        return;
    }

    string trackingNumber;
    cout << "Enter your Tracking ID: ";
    cin >> trackingNumber;

    // checking if shipment exists
    bool found = false;
    for (int i = 0; i < shipmentCount; i++) {
        if (trackingIDs[i] == trackingNumber) {
            cout << "Shipment found. Please enter the new details:" << endl;

            if (shipmentStatus[i] == "Pending") {
                found = true;
                cin.ignore(numeric_limits<streamsize>::max(), '\n');

                // updating all the details except tracking number
                cout << "Enter new sender name: ";
                getline(cin, senderNames[i]);

                cout << "Enter new receiver name: ";
                getline(cin, receiverNames[i]);

                cout << "Enter new receiver address: ";
                getline(cin, receiverAddresses[i]);

                cout << "Enter new receiver city: ";
                getline(cin, receiverCities[i]);

                do {
                    cout << "Enter new weight (in kg): ";
                    cin >> weights[i];
                    if (weights[i] <= 0 || cin.fail()) {  // handling excceptions for weight input
                        cout << "\033[31mWeight must be a positive number. Please try again.\033[0m" << endl;
                        cin.clear();
                        cin.ignore(numeric_limits<streamsize>::max(), '\n'); // discard invalid input
                        weights[i] = -1; // reset weight to stay in loop
                    }
                } while (weights[i] <= 0);

                cin.ignore(numeric_limits<streamsize>::max(), '\n'); // flush leftover \n due to cin.

                cout << "Is the shipment fragile? (Yes/No): ";
                cin >> fragilities[i];

                cout << "Do you want fast delivery? (Yes/No): ";
                cin >> fastDeliveries[i];

                cout << "\nShipment details updated successfully!" << endl;
                shipmentCosts[i] = costCalculator(senderCities[i], receiverCities[i], weights[i], fragilities[i], fastDeliveries[i]);
                cout << "\nEstimated Cost: " << shipmentCosts[i] << " PKR" << endl;

            }
            else {
                cout << "\n\033[31mYour Shipment can't be edited because it has already been shipped.\033[0m" << endl;
            }
        }
    }

    if (!found) {
        cout << "\n\033[31mNo shipment found for this tracking ID.\033[0m" << endl;
    }
    getch();
}
```

Output

```
==========  EDIT SHIPMENT DETAILS  ==========
Enter your Tracking ID: MAC2
Shipment found. Please enter the new details:
Enter new sender name: Naveed Akhtar
Enter new receiver name: Hania Amir
Enter new receiver address: DHA, Rayaa
Enter new receiver city: Lahore
Enter new weight (in kg): 8
Is the shipment fragile? (Yes/No): No
Do you want fast delivery? (Yes/No): Yes

Shipment details updated successfully!

Estimated Cost: 1400 PKR
```

11. Cancel Shipment

This function cancels the shipment and update the shipment status.

```
// function to cancel a shipment:
void cancelShipment() {

    cout<< "\n\033[34m==========  CANCEL YOUR SHIPMENT  =========  ========\033[0m\n";

    // checking if there are shipments to cancel
    if (shipmentCount == 0) {
        cout << "\033[31mNo shipments available to cancel.\033[0m" << endl;
        getch();
        return;
    }

    string trackingNumber;
    cout << "Enter your tracking number: ";
    cin >> trackingNumber;

    // checking if shipment exists
    bool found = false;
    for (int i = 0; i < shipmentCount; i++) {
        if (trackingIDs[i] == trackingNumber) {
            if (shipmentStatus[i] == "Pending") {
                shipmentStatus[i] = "Cancelled";  // updating status to cancelled
                cout << "\nYour shipment with \033[1m tracking ID " << trackingIDs[i] <<"\033[0m has been cancelled. Please collect your parcel from the staff." << endl;
            }
            else {
                cout << "\n\033[31mYour Shipment with \033[1m tracking ID " << trackingIDs[i] <<"\033[0m can't be cancelled because it has already been shipped.\033[0m" << endl;
            }
            found = true;
        }
    }

    if (!found) {
        cout << "\n\033[31mNo shipment found with the given tracking number.\033[0m" << endl;
    }

    getch();
}
```

Output

```
==========  CANCEL YOUR SHIPMENT  ==========
Enter your tracking number: MAC2

Your shipment with  tracking ID MAC2 has been cancelled. Please collect your parcel from the staff.
```

Before Editing and Cancelling

```
Consigner Shipment 3:
-----------------------------------------------------
  Tracking Number:    MAC2
  Sender Name:        Noman Iqbal
  Receiver Name:      Ayesha Noor
  Sender's City:      Rawalpindi
  Receiver's City:    Rawalpindi
  Receiver's Address: Satellite Town
  Weight:             5 kg
  Fragile:            No
  Fast Delivery:      No
  Estimated Cost:     600 PKR
  Shipment Status:    Pending
-----------------------------------------------------
```

After Editing and Cancelling

```
Consigner Shipment 3:
-----------------------------------------------------
  Tracking Number:    MAC2
  Sender Name:        Naveed Akhtar
  Receiver Name:      Hania Amir
  Sender's City:      Rawalpindi
  Receiver's City:    Lahore
  Receiver's Address: DHA, Rayaa
  Weight:             8 kg
  Fragile:            No
  Fast Delivery:      Yes
  Estimated Cost:     1400 PKR
  Shipment Status:    Cancelled
-----------------------------------------------------
```

12. Report Damage

This function checks if the shipment has been delivered and then takes the report. It stores responses in a global array.

```
string damages[50]; // to store damages (if any).

// function for consigner to report damage or check status:
void reportDamage() {

    cout<< "\n\033[34m==========  REPORT A DAMAGE  ==========\033[0m\n";

    if (shipmentCount == 0) {
        cout << "\n\033[31mNo shipments available.\033[0m" << endl;
        getch();
        return;
    }

    string trackingNumber;
    cout << "Enter your tracking ID to report/check damage: ";
    cin >> trackingNumber;

    bool found = false;

    // loop through all shipments to find the matching tracking ID
    for (int i = 0; i < shipmentCount; i++) {
        if (trackingIDs[i] == trackingNumber) {
            found = true;
            if (shipmentStatus[i] == "Delivered")
            {
                // If damage not reported yet, record it
                if (damages[i].empty()) {
                    damages[i] = "No Response Yet"; // mark as reported by consigner
                    cout << "\nYour damage has been reported successfully. "
                         << "Our team will contact you shortly." << endl;
                } else {
                    // If company hasn't handled it yet
                    if (damages[i] == "No Response Yet") {
                        cout << "\nYour damage report is still being processed. "
                             << "No response yet. Please wait for our call." << endl;
                    }
                    // If company has marked it as reported
                    else if (damages[i] == "Reported") {
                        cout << "\nYour damage has been reported successfully. "
                             << "Our team will contact you shortly." << endl;
                    }
                }
            }
            else {
                cout << "\n\033[0mReport Unsuccessful because your shipment is not delivered yet.\033[0m";
            }

            getch(); // pause so user can read
            break;
        }
    }

    if (!found) {
        cout << "\n\033[31mNo shipment found for this tracking ID.\033[0m" << endl;
        getch();
    }
}
```

Output

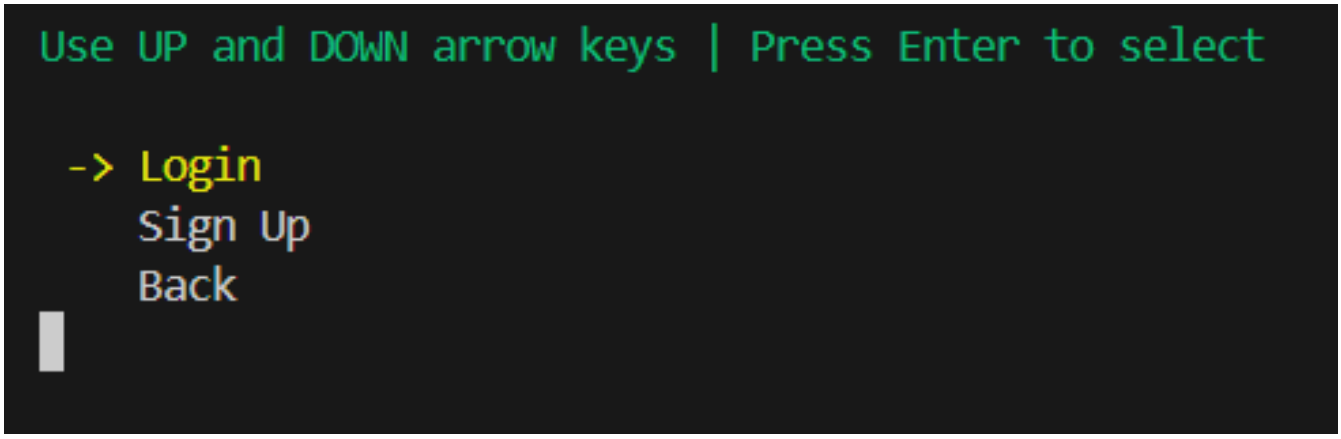
```
==========  REPORT A DAMAGE  ==========
Enter your tracking ID to report/check damage: MAC0

Your damage has been reported successfully. Our team will contact you shortly.
```

13. Company Login / Sign-Up Menu

This function displays a menu for company users to choose between login, sign-up, or going back, and navigates accordingly using the arrow-key menu system.

```cpp
// function for company login/sign up menu
void companyLoginMenu() {
    string companyAccounts[3] = {"Login", "Sign Up", "Back"};

    while (true) {
        int choice = menuNavigator(companyAccounts, 3);

        if (choice == 0) {
            if (login()) {
                cout << "Successfully logged in as a company!" << endl;
                getch();
                companyMenu();
                break;
            }
        }

        else if (choice == 1) {
            signUp();
        }
        else if (choice == 2) {
            return;
        }
    }
}

// dynamic memory allocations for sroting credentials:
    int size = 50;
    string usernames[50];
    string passwords[50];
    int idx = 0;
```

Output



14. Company Sign-Up Function

This function registers a new company account by validating username uniqueness, confirming password matching, and verifying an authorization code before storing credentials.

```cpp
// funtion to sign up for a new account:
void signUp() {
    system("cls");

    setColor(1);
    cout<< "\n==============  SIGN-UP  ================\n";
    setColor(7);

    // declarations:
    string username, pass, confirmPass;

    // autherization code.
    const int authCode = 101;
    int enteredCode;

    // I/O & checking if username already exists

    bool isExists;
    do {
        isExists = false;
        cin.ignore(numeric_limits<streamsize>::max(), '\n');

        cout << "Username: ";
        getline(cin, username);

        for (int i = 0; i <= idx; i++)
        {
            if (usernames[i] == username)
            {
                isExists = true;
                cout<<"\033[31mUsername already exists! Try a different one.\033[0m\n";
                break;
            }
        }
    } while(isExists);

    cout << "Password: ";
    cin >> pass;

    // checking similarity for confirm password:
    do
    {
        cout<< "Confirm Password: ";
        cin>> confirmPass;
        if (confirmPass != pass)
        {oes not match. Try again!\033[0m\n";
        }
    } while (confirmPass != pass);
```

```cpp
  // validation check for authourization code:
    do
    {
        cout << "Authorization Code: ";
        cin >> enteredCode;
        if (enteredCode != authCode)
        {
            cout<<"\033[31mIncorrect code! Ask the authorities for the code.\033[0m\n";
        }
    } while (enteredCode != authCode);

    // store credentials:
    usernames[idx] = username;
    passwords[idx] = pass;
    idx++;


    cout<<"Account has been created successfully!" << endl;
    getch();
}
            cout<<"\033[31mPassword d
```

Output

```
==============  SIGN-UP  ================
Username: Ahsan Mustafa
Password: 676767
Confirm Password: 676767
Authorization Code: 101
Account has been created successfully!
```

15. Company Login Function

This function authenticates company staff by verifying entered credentials against stored usernames and passwords before granting access.

```cpp

// function for logging in as a company staff:
bool login() {
    system("cls");
    setColor(1);
    cout<< "\n==============  LOGIN  ================\n";
    setColor(7);

    string username, password;
    cin.ignore(numeric_limits<streamsize>::max(), '\n');

    // taking inputs from user
    cout << "Enter Username: ";
    getline(cin, username);

    cout << "Enter Password: ";
    cin >> password;

    // checking credentials
    for (int i = 0; i < idx; i++)
    {
        if (username == usernames[i] && password == passwords[i])
        {
            return true;
        }
    }

    cout << "\033[31mInvalid username or password. Please try again.\033[0m" << endl;
    getch();
    return false;
}


```

Output

```
==============  LOGIN  ================
Enter Username: Ahsan Mustafa
Enter Password: 676767
Successfully logged in as a company!
```

16. Company Dashboard & View All Shipments

This function displays the company menu and allows staff to view all shipments in the system with complete shipment details and current status.

```
// function for company menu:
void companyMenu() {
    string companyMenuOptions[7] = {"View all Shipments", "Add New Vehicles", "Assign Vehicles", "Update Shipment Status",
                                    "Handle Complaints", "Back"};

    while (true) {
        setColor(1);
        cout <<"=====================================================================\n"
            << "||                          COMPANY  MENU                          ||\n"
            << "=====================================================================\n";
        setColor(7);
        int choice = menuNavigator(companyMenuOptions, 6);
        system("cls");

        if (choice == 0) {
            viewAllShipments();
        }
        else if (choice == 1) {
            addVehicles();
        }
        else if (choice == 2) {
            assignVehicles();
        }
        else if (choice == 3) {
            updateShipmentStatus();
        }
        else if (choice == 4) {
            handleComplaints();
        }
        else {
            break;
        }
    }
}

void viewAllShipments() {
    cout<< "\n\033[34m==============  VIEW ALL SHIPMENTS  ================\033[0m\n";
    if (shipmentCount == 0) {
            cout << "\033[31mNo shipments available.\033[0m" << endl;
            getch();
            return;
    }
```

```

    for(int i = 0; i<shipmentCount; i++){
        cout<< "Shipment " <<i+1 <<":" <<endl;
        cout << "-----------------------------------------------------" << endl;

        cout << left; // align text to the left

        cout << "  " << setw(20) << "Tracking Number:" << trackingIDs[i] << endl;
        cout << "  " << setw(20) << "Sender Name:" << senderNames[i] << endl;
        cout << "  " << setw(20) << "Receiver Name:" << receiverNames[i] << endl;
        cout << "  " << setw(20) << "Sender's City:" << senderCities[i] << endl;
        cout << "  " << setw(20) << "Receiver's City:" << receiverCities[i] << endl;
        cout << "  " << setw(20) << "Weight:" << weights[i] << " kg" << endl;
        cout << "  " << setw(20) << "Fragile:" << fragilities[i] << endl;
        cout << "  " << setw(20) << "Fast Delivery:" << fastDeliveries[i] << endl;
        cout << "  " << setw(20) << "Estimated Cost:" << shipmentCosts[i] << " PKR" << endl;
        cout << "  " << setw(20) << "Shipment Status:" << "\033[1m" << shipmentStatus[i] << "\033[0m" << endl;

        cout << "-----------------------------------------------------\n" << endl;
    }

    getch();
}
```

Output

```
Use UP and DOWN arrow keys | Press Enter to select

 -> View all Shipments
    Add New Vehicles
    Assign Vehicles
    Update Shipment Status
    Handle Complaints
    Back
```

17. Add Vehicles

This function and respective arrays allows the company to register delivery vehicles (up to five), ensuring valid input and maintaining vehicle count.

```cpp

    // array to store vehicle Reg. no (max 5 vehicles):
    string vehicles[5];
    // keeps track of how many vehicles have been added:
    int filledSlots = 0;
    // stores which vehicle each shipment is assigned to (one entry per shipment):
    string shipmentVehicle[50];

// function to add a new vehicle:
void addVehicles() {
    cout<< "\n\033[34m==============  ADD A NEW VEHICLE  ================\033[0m\n";
    // check if we already have 5 vehicles:
    if (filledSlots >= 5) {
        cout << "Sufficient vehicles have already been added for your shipments." << endl;
        getch();
        return;
    }

    // flush any leftover input from previous cin operations
    cin.ignore(numeric_limits<streamsize>::max(), '\n');

    // ask the user to input a vehicle ID
    cout << "Add a new vehicle (e.g., ABC-123): ";
    getline(cin, vehicles[filledSlots]);

    // input validation:
    if (vehicles[filledSlots].empty()) {
        cout << "\033[31mVehicle name cannot be empty!\033[0m" << endl;
        getch();
        return;
    }

    filledSlots++;
    cout << "Vehicle added successfully!" << endl;
    getch();
}

```

Output

```
==============  ADD A NEW VEHICLE  =================
Add a new vehicle (e.g., ABC-123): AKJ243
Vehicle added successfully!
```

18. Assign Vehicles to Shipments

This function automatically assigns pending shipments to available vehicles while balancing load and limiting each vehicle to a maximum number of shipments.

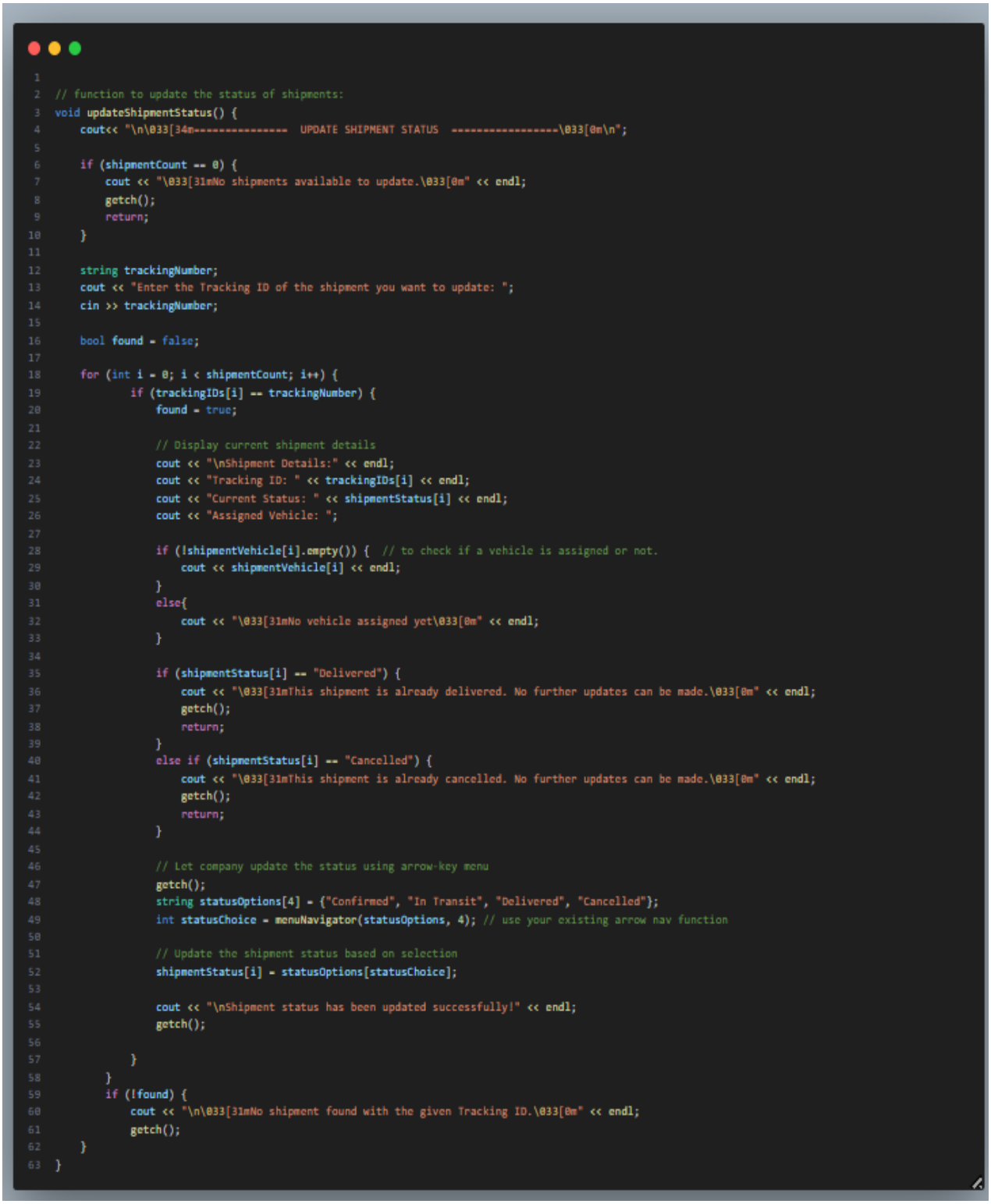
```cpp
// function to automatically assign shipments to vehicles:
void assignVehicles() {
    cout<< "\n\033[34m==========  ASSIGN VEHICLES  ==========\033[0m\n";
    if (filledSlots == 0)   // if we have not added any vehicle yet
    {
        cout << "\033[31mNo vehicles available. Please add vehicles first.\033[0m" << endl;
        getch();
        return;
    }

    int vehicleIndex = 0; // keeps track of which vehicle to assign next
    int vehicleLoad[5] = {0}; // tracks how many shipments have been assigned to each vehicle
    int assignedCount = 0; // counter for how many shipments got assigned

    for (int i = 0; i < shipmentCount; i++) {
        // only assign shipments that are pending
        if (shipmentStatus[i] == "Pending") {

            // find the next vehicle that has less than 10 shipments
            int attempts = 0;
            while (vehicleLoad[vehicleIndex] >= 10 && attempts < filledSlots) {
                vehicleIndex++;
                if (vehicleIndex >= filledSlots) vehicleIndex = 0;
                attempts++;
            }

            // if all vehicles are full (each has 10 shipments), stop assigning
            if (vehicleLoad[vehicleIndex] >= 10) {
                cout << "\033[31mAll vehicles are full. Cannot assign more shipments.\033[0m" << endl;
                break;
            }

            shipmentVehicle[i] = vehicles[vehicleIndex]; // assign current vehicle
            vehicleLoad[vehicleIndex]++; // increment this vehicle's shipment count
            vehicleIndex++; // move to next vehicle for the next shipment
            if (vehicleIndex >= filledSlots){
                vehicleIndex = 0;
            }  // loop back if needed
            assignedCount++;
        }
    }

    // if no shipments were available to assign
    if (assignedCount == 0) {
        cout << "\033[31mNo shipments available for assignment.\033[0m" << endl;
    }
    else {
        // display the vehicle assignments for the company to see:
        cout << "Shipments have been assigned to vehicles automatically:\n\n";

        for (int i = 0; i < shipmentCount; i++) {
            if (!shipmentVehicle[i].empty()) {
                cout << "Shipment " << trackingIDs[i]
                    << " -> Vehicle " << shipmentVehicle[i] << endl;
            }
        }
    }

    getch();
}
```

Output

```
==========  ASSIGN VEHICLES  ==========
Shipments have been assigned to vehicles automatically:

Shipment MAC0 -> Vehicle AKJ243
Shipment MAC1 -> Vehicle ANH981
```

19. Update Shipment Status

This function enables company staff to update shipment status using an interactive menu, while preventing updates to delivered or cancelled shipments.



```
// function to update the status of shipments:
void updateShipmentStatus() {
    cout<< "\n\033[34m==============  UPDATE SHIPMENT STATUS  ================\033[0m\n";

    if (shipmentCount == 0) {
        cout << "\033[31mNo shipments available to update.\033[0m" << endl;
        getch();
        return;
    }

    string trackingNumber;
    cout << "Enter the Tracking ID of the shipment you want to update: ";
    cin >> trackingNumber;

    bool found = false;

    for (int i = 0; i < shipmentCount; i++) {
            if (trackingIDs[i] == trackingNumber) {
                found = true;

                // Display current shipment details
                cout << "\nShipment Details:" << endl;
                cout << "Tracking ID: " << trackingIDs[i] << endl;
                cout << "Current Status: " << shipmentStatus[i] << endl;
                cout << "Assigned Vehicle: ";

                if (!shipmentVehicle[i].empty()) {  // to check if a vehicle is assigned or not.
                    cout << shipmentVehicle[i] << endl;
                }
                else{
                    cout << "\033[31mNo vehicle assigned yet\033[0m" << endl;
                }

                if (shipmentStatus[i] == "Delivered") {
                    cout << "\033[31mThis shipment is already delivered. No further updates can be made.\033[0m" << endl;
                    getch();
                    return;
                }
                else if (shipmentStatus[i] == "Cancelled") {
                    cout << "\033[31mThis shipment is already cancelled. No further updates can be made.\033[0m" << endl;
                    getch();
                    return;
                }

                // Let company update the status using arrow-key menu
                getch();
                string statusOptions[4] = {"Confirmed", "In Transit", "Delivered", "Cancelled"};
                int statusChoice = menuNavigator(statusOptions, 4); // use your existing arrow nav function

                // Update the shipment status based on selection
                shipmentStatus[i] = statusOptions[statusChoice];

                cout << "\nShipment status has been updated successfully!" << endl;
                getch();

            }
        }
        if (!found) {
            cout << "\n\033[31mNo shipment found with the given Tracking ID.\033[0m" << endl;
            getch();
    }
}
```

Output

```
=============== UPDATE SHIPMENT STATUS ================
Enter the Tracking ID of the shipment you want to update: MAC0

Shipment Details:
Tracking ID: MAC0
Current Status: Pending
Assigned Vehicle: AKJ243
Use UP and DOWN arrow keys | Press Enter to select

    In Transit
  -> Delivered
    Cancelled

Shipment status has been updated successfully!
```

Updated Shipment Status View

```
============== VIEW ALL SHIPMENTS ================
Shipment 1:
--------------------------------------------------
  Tracking Number:    MAC0
  Sender Name:        Ahsan Mustafa
  Receiver Name:      Moghees Mohiuddin
  Sender's City:      Fort Abbas
  Receiver's City:    Lahore
  Weight:             10 kg
  Fragile:            No
  Fast Delivery:      Yes
  Estimated Cost:     1700 PKR
  Shipment Status:    Delivered
--------------------------------------------

Shipment 2:
--------------------------------------------
  Tracking Number:    MAC1
  Sender Name:        Kamran Siddique
  Receiver Name:      Zain Abbas
  Sender's City:      Sheikhupura
  Receiver's City:    Lahore
  Weight:             25 kg
  Fragile:            Yes
  Fast Delivery:      No
  Estimated Cost:     3820 PKR
  Shipment Status:    In Transit
--------------------------------------------------

Shipment 3:
--------------------------------------------------
  Tracking Number:    MAC2
  Sender Name:        Naveed Akhtar
  Receiver Name:      Hania Amir
  Sender's City:      Rawalpindi
  Receiver's City:    Lahore
  Weight:             8 kg
  Fragile:            No
  Fast Delivery:      Yes
  Estimated Cost:     1400 PKR
  Shipment Status:    Cancelled
--------------------------------------------------
```

20. Handle Complaints

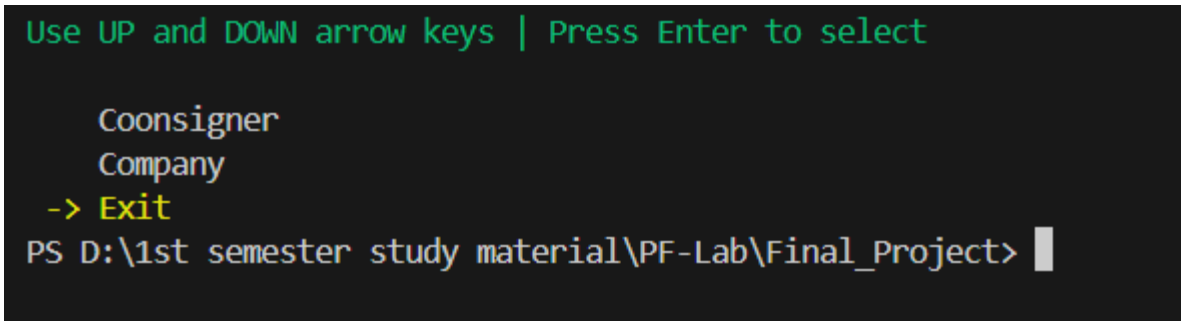
This function allows company staff to view pending damage complaints and mark selected complaints as handled using arrow-key navigation.

```cpp
// Function for company to handle complaints
void handleComplaints() {
    cout<< "\n\033[34m==========  HANDLE COMPLAINTS  ==========\033[0m\n";
    // Count how many complaints exist
    int complaintCount = 0;
    for (int i = 0; i < shipmentCount; i++) {
        if (!damages[i].empty() && damages[i] == "No Response Yet") complaintCount++;
    }

    if (complaintCount == 0) {
        cout << "\033[31mNo complaints available at the moment.\033[0m" << endl;
        getch();
        return;
    }

    // Prepare menu for arrow-key navigation
    string complaintList[complaintCount];
    int indices[complaintCount]; // store shipment indices
    int idx = 0;

    for (int i = 0; i < shipmentCount; i++) {
        if (damages[i] == "No Response Yet") {
            complaintList[idx] = trackingIDs[i]; // show only tracking IDs
            indices[idx] = i;
            idx++;
        }
    }

    // Let company select a complaint using arrow keys
    int choice = menuNavigator(complaintList, complaintCount);
    int shipmentIndex = indices[choice];

    // Mark the selected complaint as reported
    damages[shipmentIndex] = "Reported";

    cout << "\nComplaint for Shipment " << trackingIDs[shipmentIndex]
        << " has been marked as reported." << endl;
    getch();
}
```

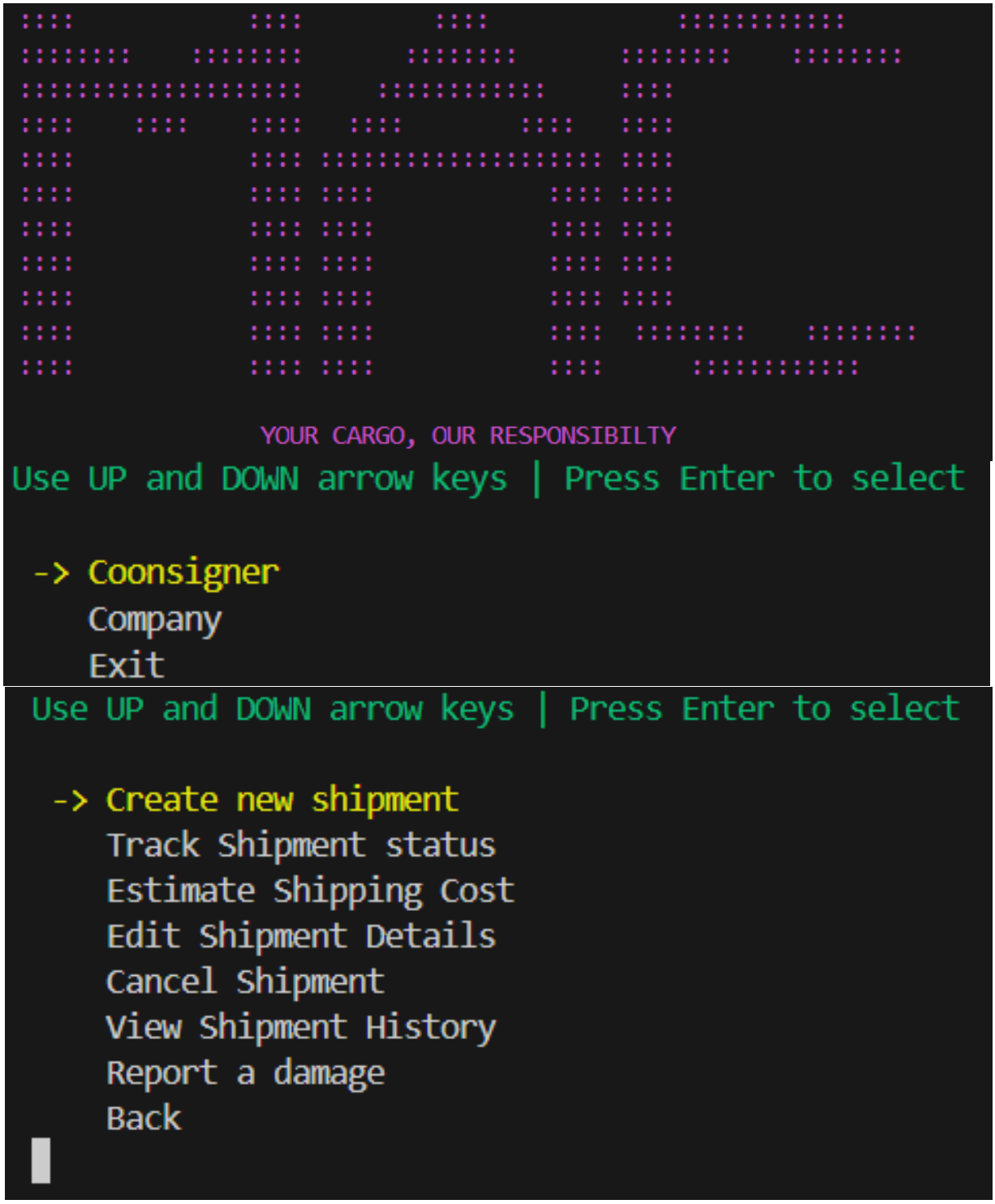
Output

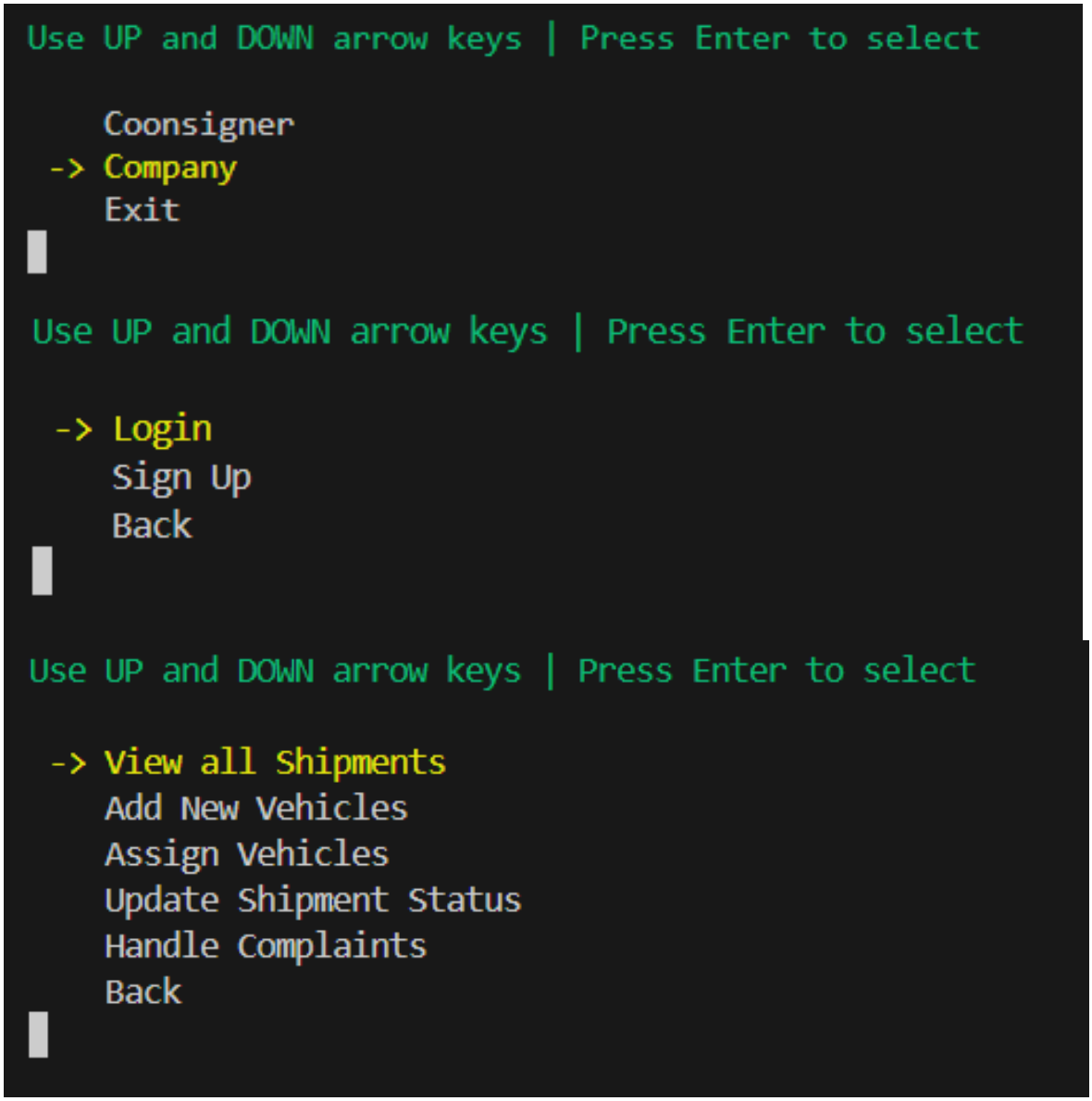
```
Use UP and DOWN arrow keys | Press Enter to select

 -> MAC0

Complaint for Shipment MAC0 has been marked as reported.
```

21. Exit Program



Major Ouputs




Use UP and DOWN arrow keys | Press Enter to select
Coonsigner
-> Company
Exit
Use UP and DOWN arrow keys | Press Enter to select
-> Login
Sign Up
Back
Use UP and DOWN arrow keys | Press Enter to select
-> View all Shipments
Add New Vehicles
Assign Vehicles
Update Shipment Status
Handle Complaints
Back